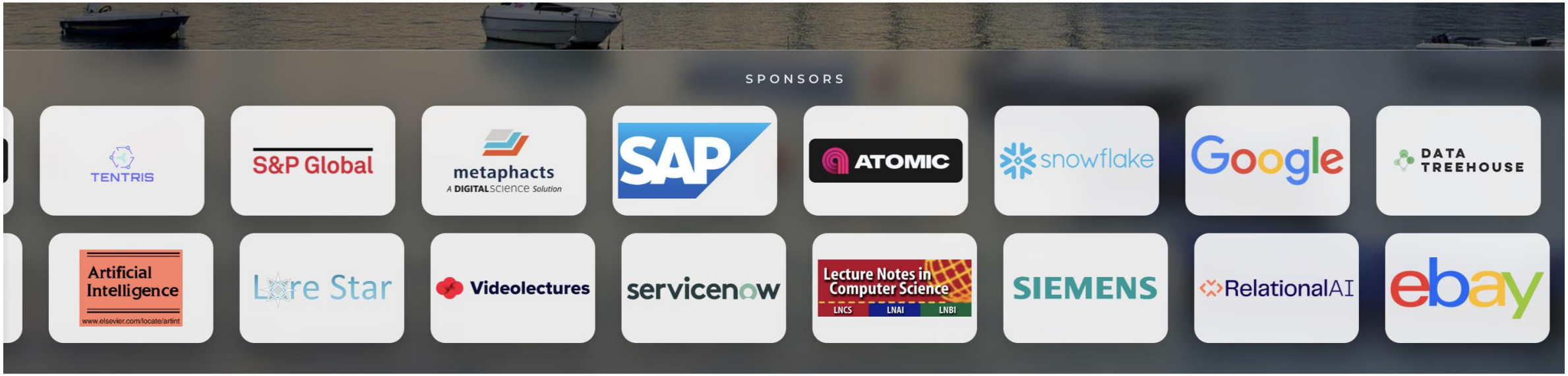



# UVic Innovation Centre

## Your launchpad for entrepreneurial success




# About ISWC 2026

## Ontologies, Knowledge Graphs, and Smarter AI

The International Semantic Web Conference (ISWC) is the premier international forum for the Semantic Web and Linked Data community. ISWC 2026 will bring together researchers, practitioners, and industry specialists to discuss, advance, and shape the future of semantic technologies. ISWC offers five exciting and fruitful days you don't want to miss every year.

**ISWC 2026** will be held as an in-person conference. We look forward to meeting you in **Bari**, **Italy**, in **October 2026**.

Project 5

Team Projects List

1. Project Definition and Team Formation 1

2. Background and Related Work 1

3. Proposal: 3-Minute Talk (3MT) 1

4. Solution Design and Implementation: Part I: Relational 1

5. Solution Design and

# 4. Solution Design and Implementation: Part I: Relational

Print Settings

Add dates and restrictions...

Add a description...

Upload / Create | Existing Activities | Bulk Edit

4 Solution Design & Implementation: Part I: Relational

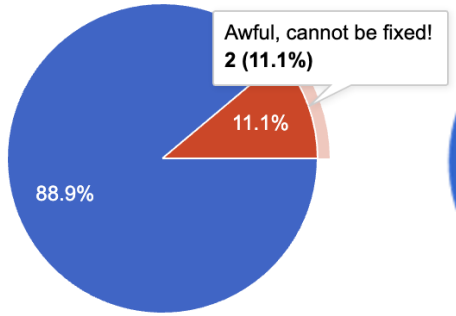
Assignment

Due November 5 at 5:00 AM

Please see the attached manual and sample submission. The sample is **for illustration only and is not a complete or authoritative example**. Please follow the manual for the actual requirements.

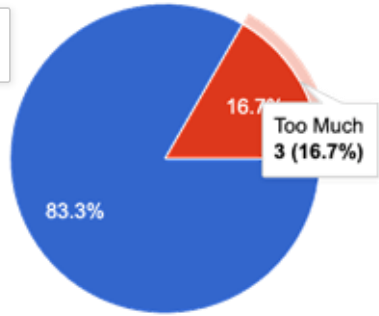
How was teaching?

18 responses



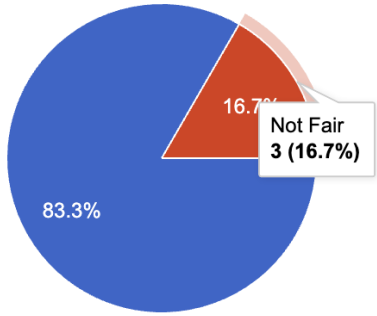

How was the load with the coursework?

18 responses



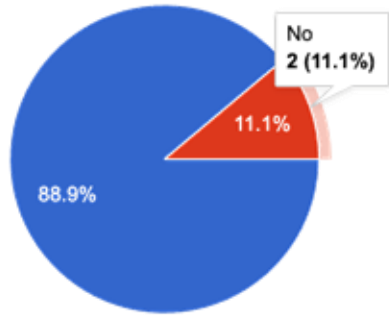

How was marking so far?

18 responses



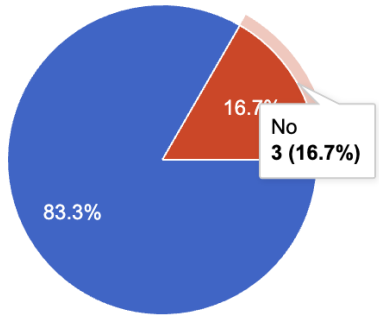

Was it worth taking this course?

18 responses



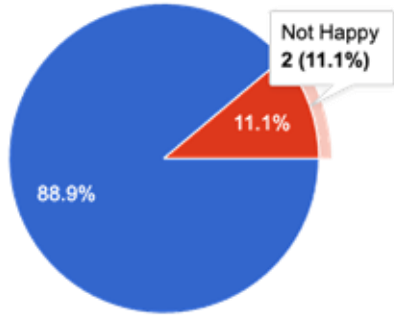

Was it worth all the coursework?

18 responses




How was the course?

18 responses




please be lenient in grading.

Grading feels mostly subjective, which can make marks seem somewhat random rather than earned: the scheme give
so expectations only become clear after grading. It also feels like things are graded that shouldn't be, such as open c

Preparing for the midterm feels a bit stressful because we don't have separate assignments to practice the lecture ma
A few optional practice questions would help me check my understanding and feel more prepared.
Before this course I never used GitHub, because we need that all the time, maybe a little introduction would have bee
There should be some assignments in the course to get a better hands on understanding instead of forcing so many
The lecture delivery is mostly just slides and no effort on the blackboard.
Just listening to a lecture for 90 mins makes anyone lose attention because it all just slides.
Definitely needs a lot of improvement in teaching style to make this dry course a bit more interesting.
Please start using blackboard to explain concepts.
Give us assignments on modern softwares and tools used to tackle the challenges faced in the course
Brightspace forum is no longer used after team formation. It would be better to make more use of it,
maybe open a Q&A section for any questions student may have, so others have same question can also see it
I don't like photography and watching movies very much, so I don't like the examples about these in the class.
By the way, the sound of the asynchronous course video is too small and the image is very blurry.
It should be a little less heavy because there are already too many subjects
More of practical needs to be learned...no flaws in lecture but still implementation with some small activities would b

However, waking up early in the morning is still hard to do.
Class could have been moved little bit later. 8:30 am is too early for me. I have to start from 6 am to reach college at
Its tough to attend classes in early morning...if teams call participation is allowed it will be great...
Class Timing is early
The timing of the class is too early especially on a MONDAY!!!!! and not enough discussion on movies.

Everything is fine!
Going very well.
Your teaching is great, professor. It's not a exaggerating, because I can see you try to convey knowledge as easy as p
so good that I can understand over 95% of your class. Only problem is this class begins early in the morning haha. E
kidding about the above, you are a great teacher!
The contents of the course is well organized, it's easy to follow.
Overall, I think the class is going well, and I like how the work builds toward the project.

5

# A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

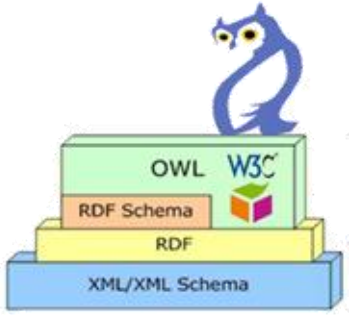
Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

Computable Entity




Data Rep. in GGG

Real World Entity

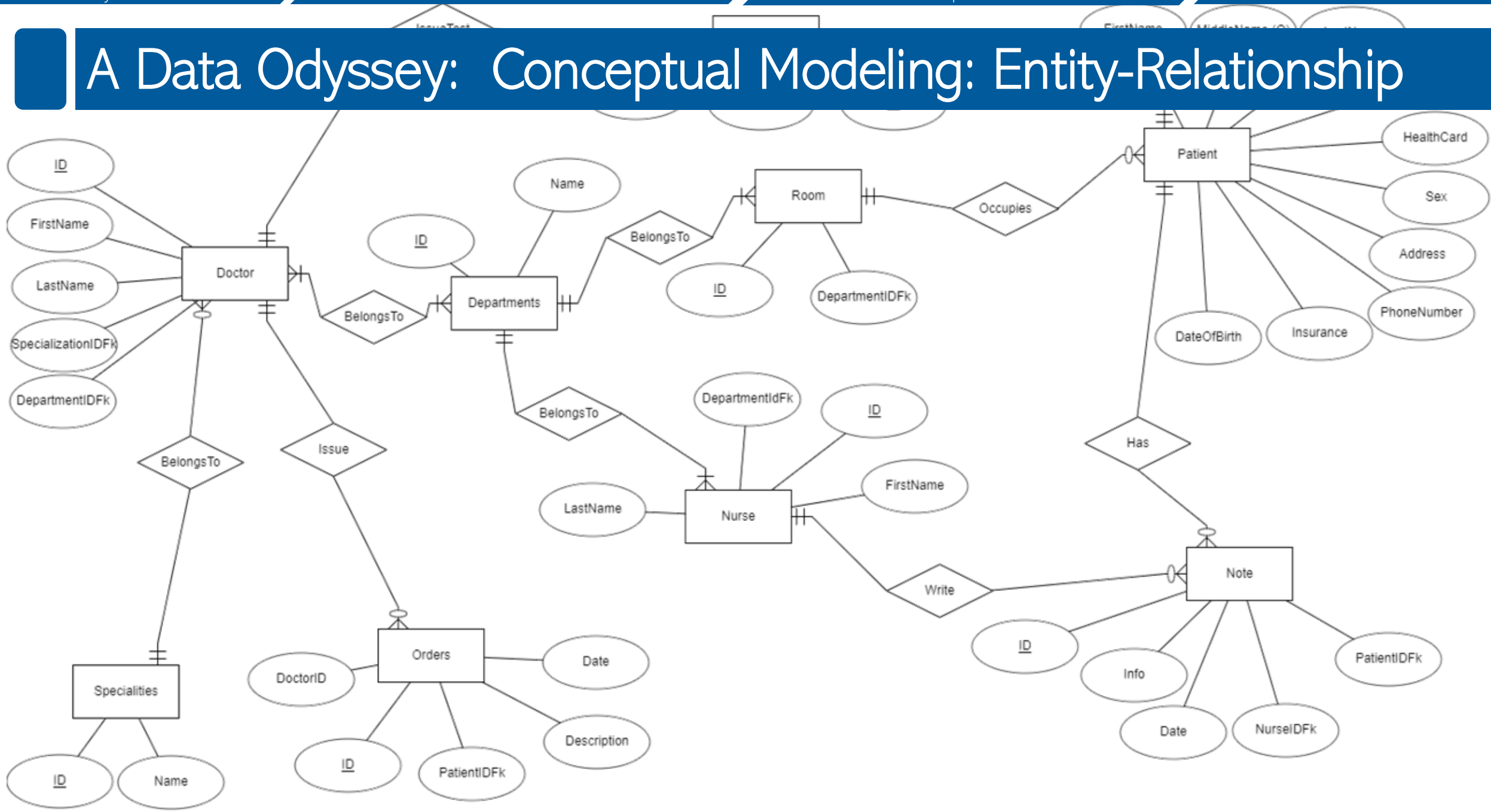
Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

| Physical Level | SPARQL

| Computable Entity

# A Data Odyssey: Conceptual Modeling: Entity-Relationship



7

# A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

Computable Entity




Data Rep. in GGG

Real World Entity

Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

| Physical Level | SPARQL

| Computable Entity

# A Data Odyssey: Logical Modeling: Relational

**R1: Patient** (Id, Number, FirstName, MiddleName, LastName, Sex, DateOfBirth, Address, Phone, HealthCardNo, InsuranceName), CK={Number} & Mandatory
**R2: Doctor** (Id, (MINC, FirstName, LastName, SpecialtyId), CK={MINC} & Mandatory, FK={SpecialtyId} & Mandatory
**R3: Room** (Id, Number, DepartmentId), CK={Number} & Mandatory, FK={DepartmentId} & Mandatory
**R4: Nurse** (Id, FirstName, LastName, DepartmentId), FK={DepartmentId} & Mandatory
**R5: Test** (Id, Name), CK={Name} & Mandatory
**R6: Order** (Id, Description, Date, DoctorId), FK={DoctorId} & Mandatory
**R7: Specialty** (Id, Name), CK={Name} & Mandatory
**R8: Department** (Id, Name), CK={Name} & Mandatory
**R9: PatientRoom** (Id, PatientId, RoomId, StartDate, EndDate), FK1={PatientId}, FK2={RoomId}, CK={PatientId, RoomId, StartDate}
**R10: PatientNurse** (Id, PatientId, NurseId, Date, Info), FK1={PatientId}, FK2={NurseId}, CK={PatientId, NurseId, Date}
**R11: DoTest** (Id, DoctorId, PatientId, TestId, Result, Date), FK1={PatientId} & Mandatory, FK2={DoctorId} & Mandatory, FK3={TestId} & Mandatory, CK={DoctorId, PatientId, TestId, Date}
**R12: DoctorDepartment** (Id, DoctorId, DepartmentId), FK1={DoctorId}, FK2={DepartmentId}, CK={DoctorId, DepartmentId}

# 9 A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

| Computable Entity




Data Rep. in GGG

Real World Entity

Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

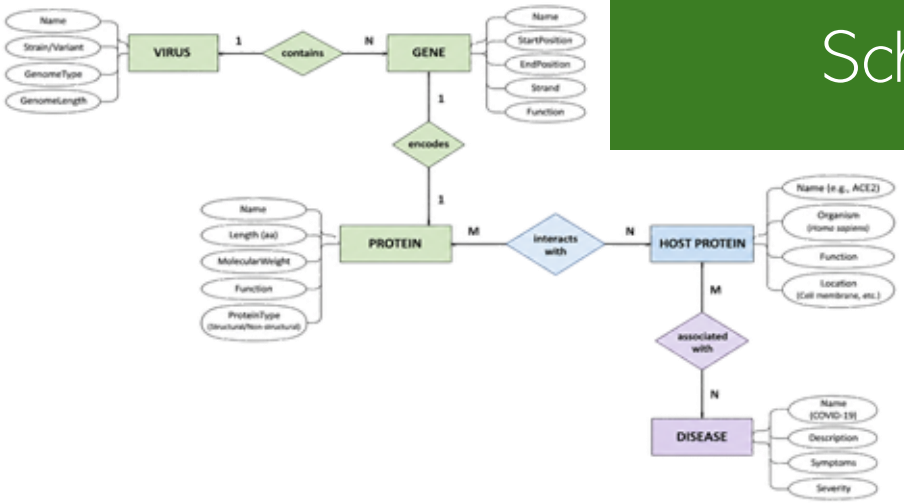
| Physical Level | SPARQL

| Computable Entity

10

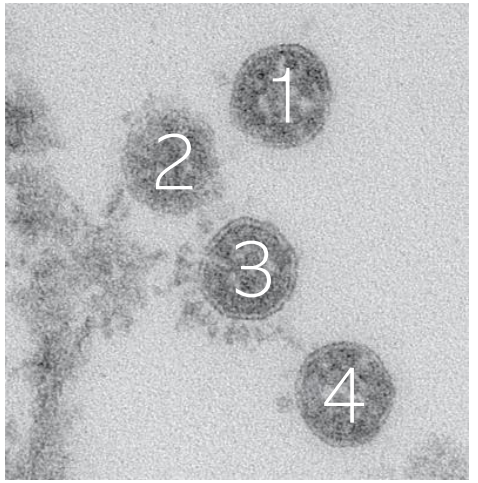
# Abstractions: Physical Representation

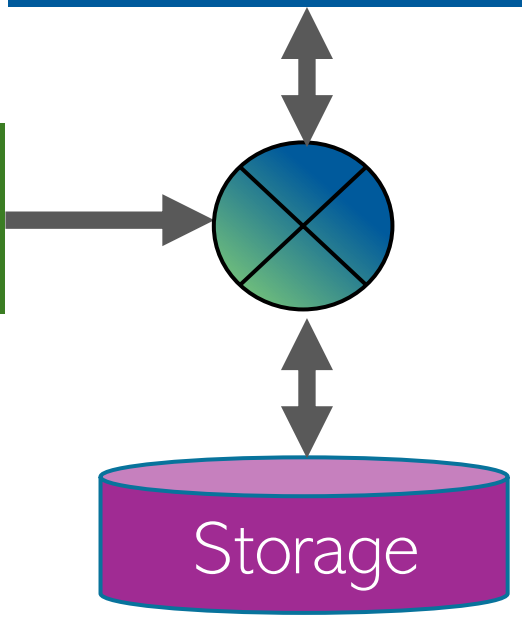
How instances of entities, values of their attributes, their relationships should be persisted on a storage.




Schema

Real-World Instances of Entity

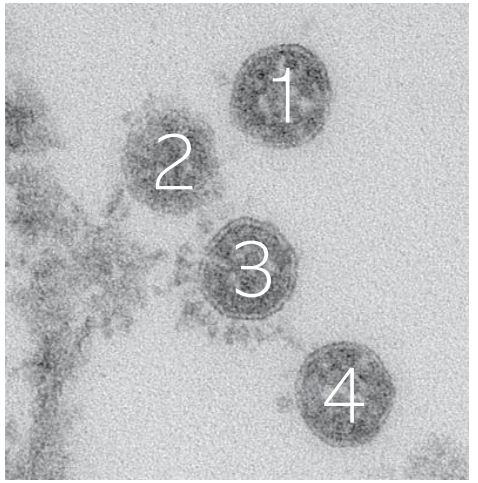


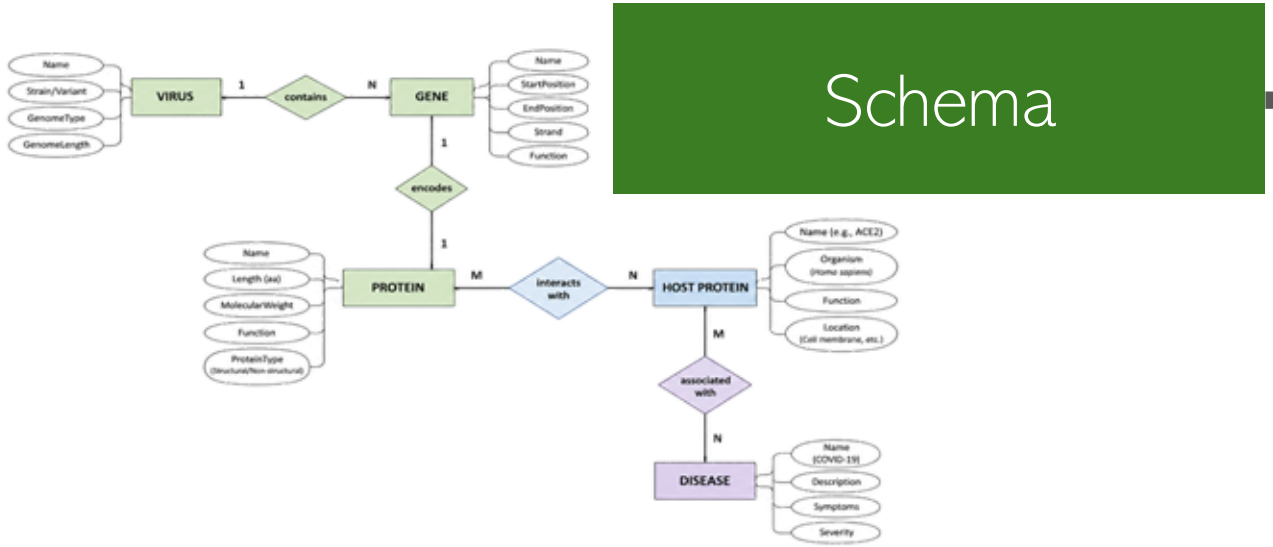



11

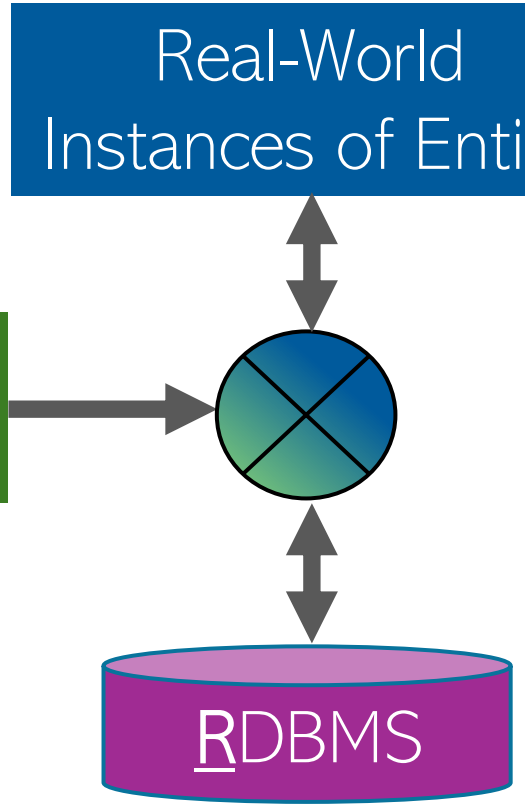
# Abstractions: Relational Database Management Systems

How instances of entities, values of their attributes, their relationships should be persisted on a storage.



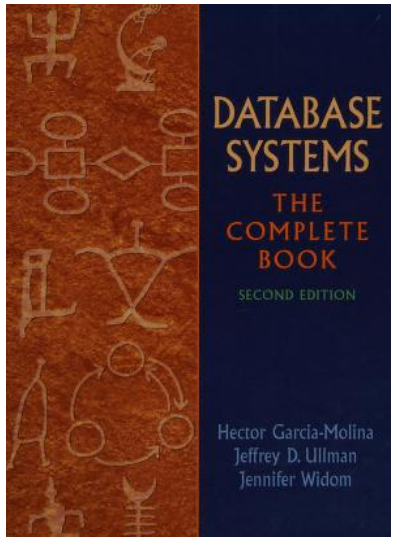

Schema





The Matrix (1999), He is the One.

# Outline: W07: Physical Modeling: SQL




CH06 (2nd Ed.)

I) Introduction

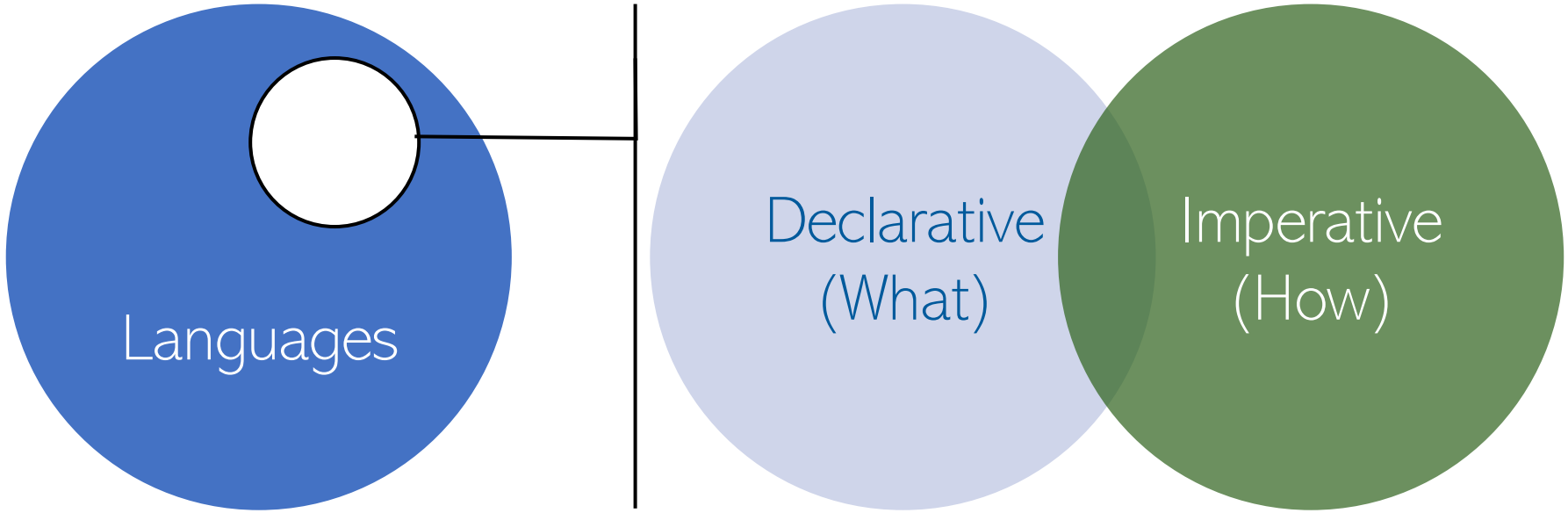
II) Data Definition Language (DDL)

III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

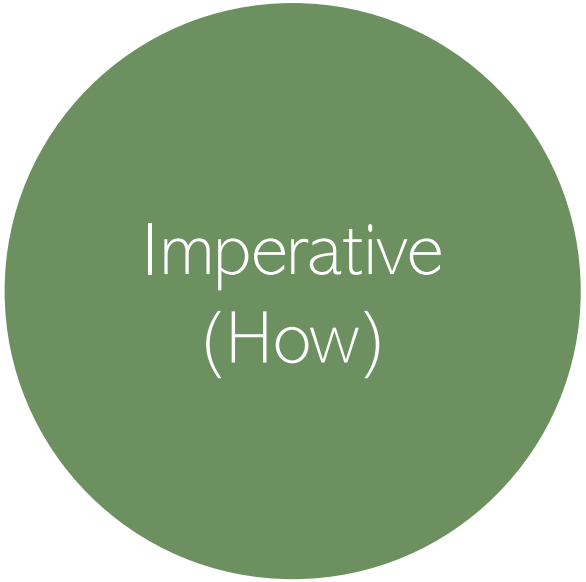
**Hands-On**

# Languages: Natural and Programming Languages



# Programming Languages: Imperative

## Explains how to do a task …




```
#include <stdio.h>
void main() {
      int n;
      scanf("%d", &n)
        int r = 1;
        for(int i = n; i > 0; i--){
            r = r * i;
        }
       printf(r);
     }
}
```

*Btw, this program is not working in practice! Why?*

# Programming Languages: Declarative

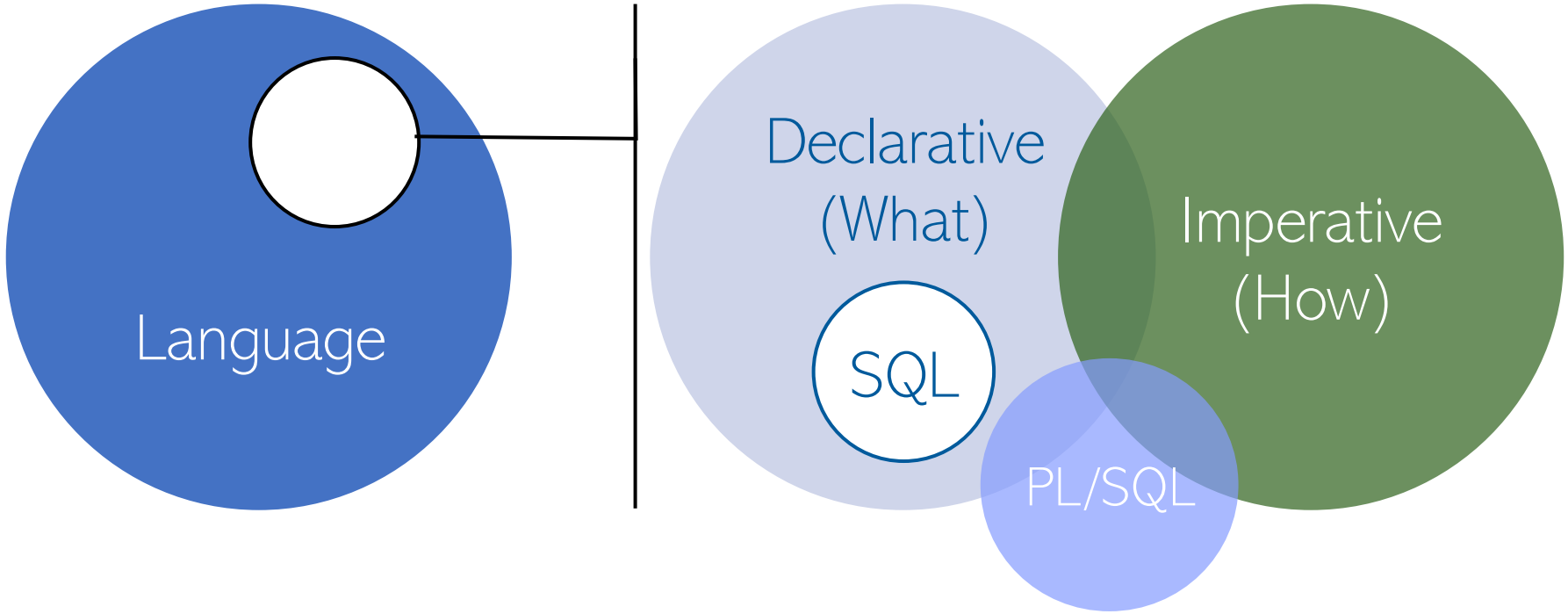
Explains what is needed, the *'system'* finds out the *'best'* way to do it.

Declarative (What)

```
SELECT FACT(n=100);
INSERT INTO Movie (...)
UPDATE Movie (...)
DELETE FROM Movie
```

Other declarative languages: HTML, CSS, LLM Prompts, …

# Programming Languages: SQL is Declarative

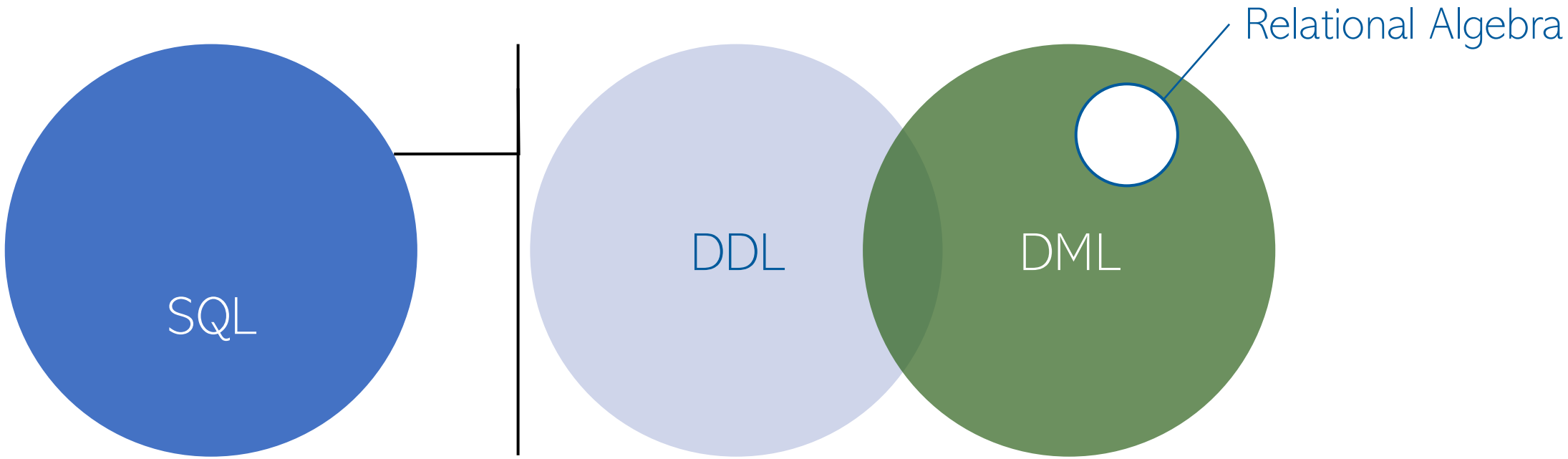


# Programming Languages: SQL is Standardized

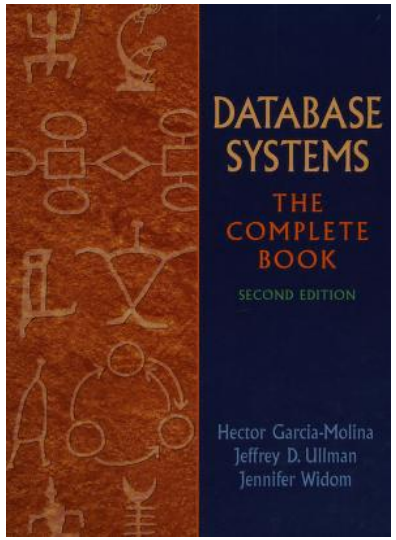
The most commonly used relational DBMS's query and modify the database through a language called SQL (sometimes pronounced "sequel"). SQL stands for "Structured Query Language." The portion of SQL that supports queries has capabilities very close to that of relational algebra. However: SQL also includes statements for modifying the database (e.g., inserting and deleting tuples from relations) and for declaring a database schema. Thus, SQL serves as both a data manipulation language (DML) and as a Data Definition Language. SQL also standardizes many other database command. There are many different dialects of SQL. First, there are three major standards. There is ANSI (American National Standards Institute) SQL and an updated standard adopted in 1992, called SQL-92 or SQL2. The recent SQL-99 (previously referred to as SQL3) standard extends SQL2 with object-relational features and a number of other new capabilities. Then, there are versions of SQL produced by the principal DBMS vendors. These all include the capabilities of the original .ITS1 standard. They also conform to a large extent to the more recent SQL2, although each has its variations and extensions beyond SQL2, including some of the features in the SQL-99 standard. Herein, we consider SQL as a stand-alone query language. SQLite does not fully support SQL-92.

19

# SQL: DDL + DML



# Outline: W07: Physical Modeling: SQL




CH06 (2nd Ed.)

I) Introduction

II) Data Definition Language (DDL)

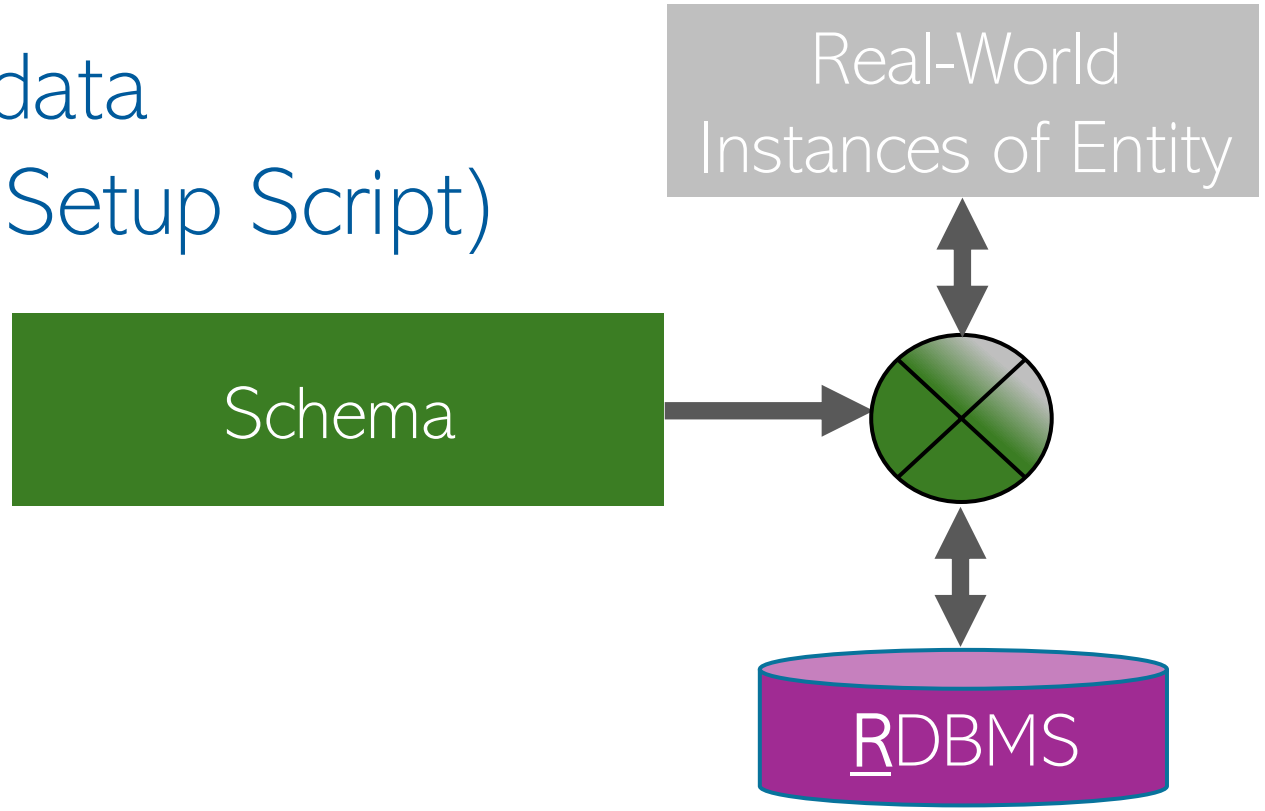
III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

Hands-On

# SQL: DDL for Database, Tables, Constraints

Data Schema
Data about Data: Meta-data
Defined at Setup Time (Setup Script)
Rarely Change



# SQL: DDL for Database, Tables, Constraints

```
CREATE DATABASE DatabaseName;
```

In SQLite > sqlite3 DatabaseName;

```
DROP DATABASE DatabaseName;
```

In SQLite > simply delete database file

```
ALTER DATABASE DatabaseName ... ;
```

In SQLite > no command!

```
RENAME DATABASE DatabaseName TO NewDatebaseName;
```

In SQLite > simply rename database file!

# SQL: DDL for Database, Tables, Constraints

```
CREATE TABLE TableName(
        ColumnName1 DataType [NULL | NOT NULL],
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
);
```

# SQL: DDL for Database, Tables, Constraints

```
ALTER TABLE TableName ADD COLUMN ColumnName DataType
ALTER TABLE TableName DROP COLUMN ColumnName
ALTER TABLE TableName RENAME COLUMN OldName TO NewName
ALTER TABLE TableName RENAME TO NewTableName
```

```
TRUNCATE TABLE TableName (empty the tables but keeps the table)
```

```
DROP TABLE TableName
```

*RDBMSs may not fully support SQL-92 → https://www.sqlite.org/omitted.html*

# SQL: DDL for Database, Tables, Constraints

`INTEGER`

`DECIMAL(i, j)` i total number of digits, j number of digits after decimal point, e.g., 12.351 ∈ `DECIMAL(5,3)`

`FLOAT | REAL` single precision real number

`DOUBLE` double precision real number

`DATE` year month day, the format can be modified, e.g., yy-mm-dd | mm-dd-yyyy

`TIME` hour:minute:second, the format can be modified, precision depends on DBMS

`DATETIME`

`TIMESTAMP`

`CHAR(n)` n character, if less, padded with space

`VARCHAR(n)` n character max, if less, no padding

`TEXT` document, article, …

`BIT` only one bit, 0 | 1

`BOOLEAN` `TRUE | FALSE`

`BLOB` large object in binary format (voice, movie, image, …)

*Data types are highly dependent on the DBMS.*
*See manual of the DBMS. e.g., https://www.sqlite.org/datatype3.html*

# SQL: DDL for Database, Tables, Constraints: Surrogate Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
);
```

# 27 SQL: DDL for Database, Tables, Constraints: Candidate Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
        CONSTRAINT UK_ColumnNames UNIQUE(candidate key columns)
);
```

# SQL: DDL for Database, Tables, Constraints: Candidate Key

```
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
```

# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
        CONSTRAINT UK_ColumnNames UNIQUE(candidate key columns)
        CONSTRAINT FK_ColumnNames FOREIGN KEY(foreign key columns)
        REFERENCES TargetTable(TargetTable's primary key columns)
);
```

# SQL: DDL for Database, Tables, Constraints: Foreign Key




R1: Movie(<u>Id</u>, Title, ReleaseDate, Language, RunningTime, DirectorId)
CK={Title}, FK={DirectorId}

R2: Director(<u>Id</u>, FirstName, LastName)
CK={FirstName, LastName}

# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
        CONSTRAINT FK_Movie_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
```

# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
        CONSTRAINT FK_Movie_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
```



# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
        CONSTRAINT FK_Movie_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
```



# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
);
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);

ALTER TABLE Movie ADD CONSTRAINT FK_Movie_DirectorId_2_Director_Id
FOREIGN KEY(DirectorId) REFERENCES Director(Id)
```



# SQL: DDL for Database, Tables, Constraints: Foreign Key




R1: Movie(Id, Title, ReleaseDate, Language, RunningTime) CK={Title}

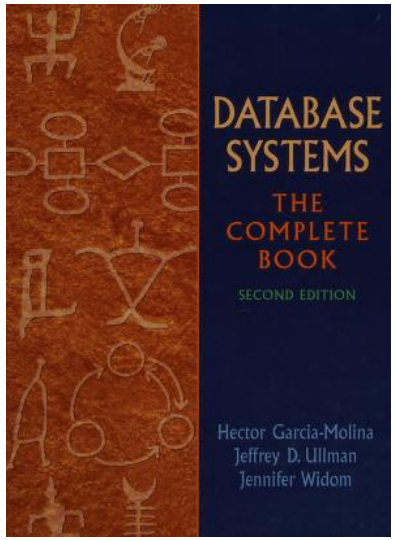
R2: Director(Id, FirstName, LastName) CK={FirstName, LastName}

R3: MovieDirector(Id, MovieId, DirectorId) $FK_1$={MovieId}, $FK_2$={DirectorId}

# SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Movie(
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Title VARCHAR(255),
            ReleaseDate DATE,
            Language VARCHAR(255),
            RunningTime INTEGER,
            CONSTRAINT UK_Title UNIQUE(Title),
);
CREATE TABLE Director(
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            FirstName VARCHAR(255) NOT NULL,
            LastName VARCHAR(255) NOT NULL,
            CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
CREATE TABLE MovieDirector(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        MovieId INTEGER NOT NULL,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT FK_MovieDirector_MovieId_2_Movie_Id FOREIGN KEY(MovieId)
        REFERENCES Movie(Id),
        CONSTRAINT FK_MovieDirector_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
```

# Outline: W07: Physical Modeling: SQL




CH06 (2nd Ed.)

I) Introduction

II) Data Definition Language (DDL)

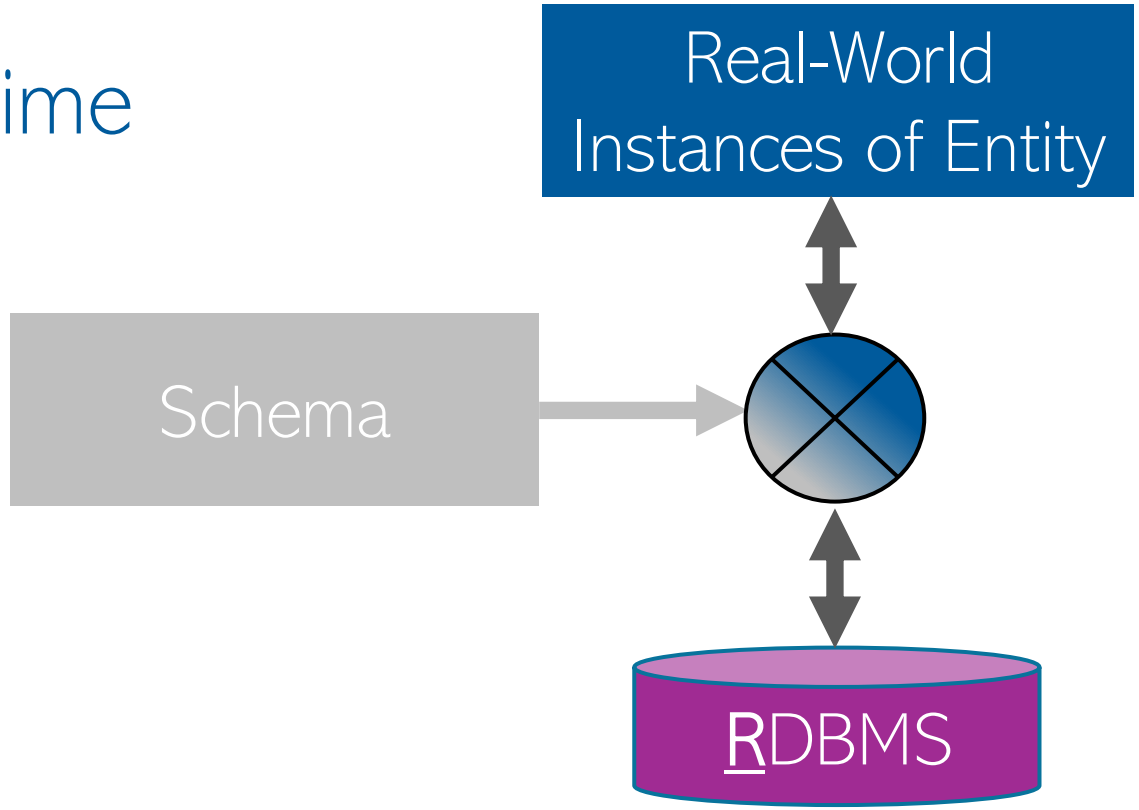
III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

Hands-On

# SQL: DML for Entity Instances (Tuples)

Actual Entities (Tuple)
Rapidly Change at Running Time
Must Conform to Schema

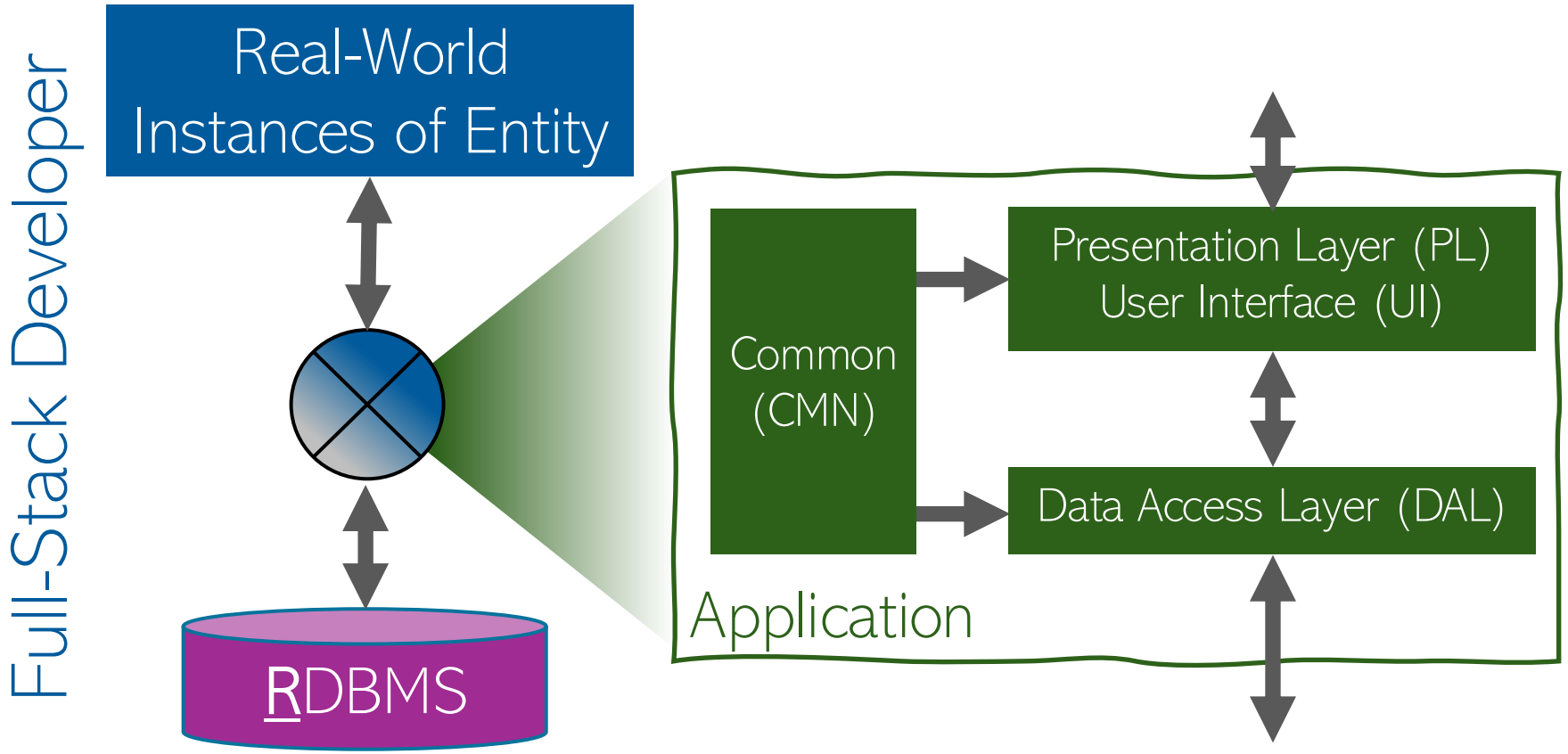



→ Data Access Layer (DAL)



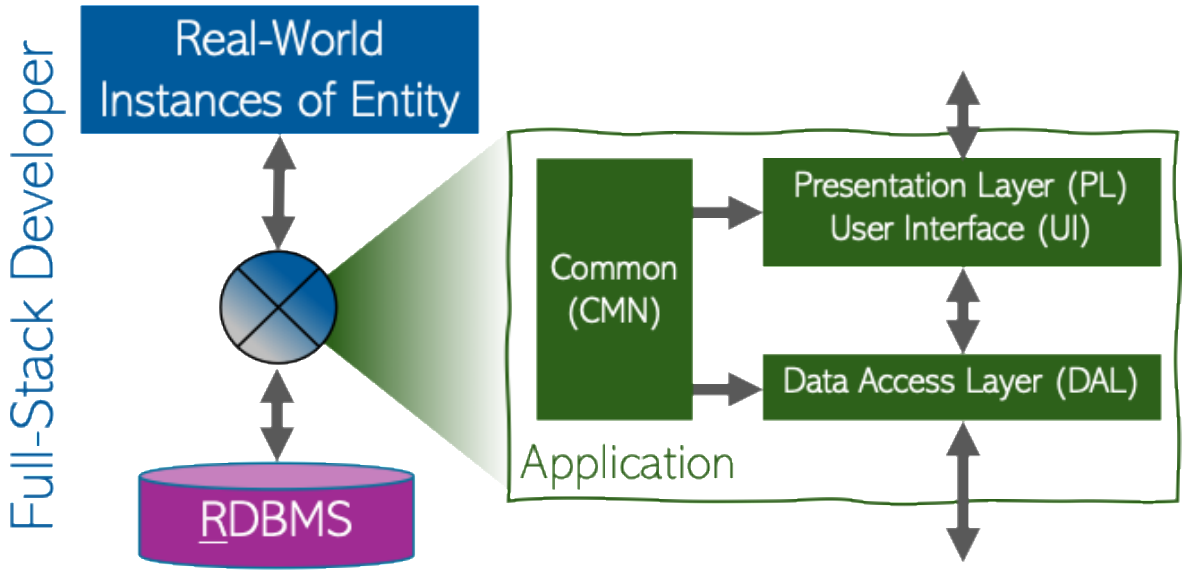
The Matrix (1999), He is the One.

# SQL: DML via DAL



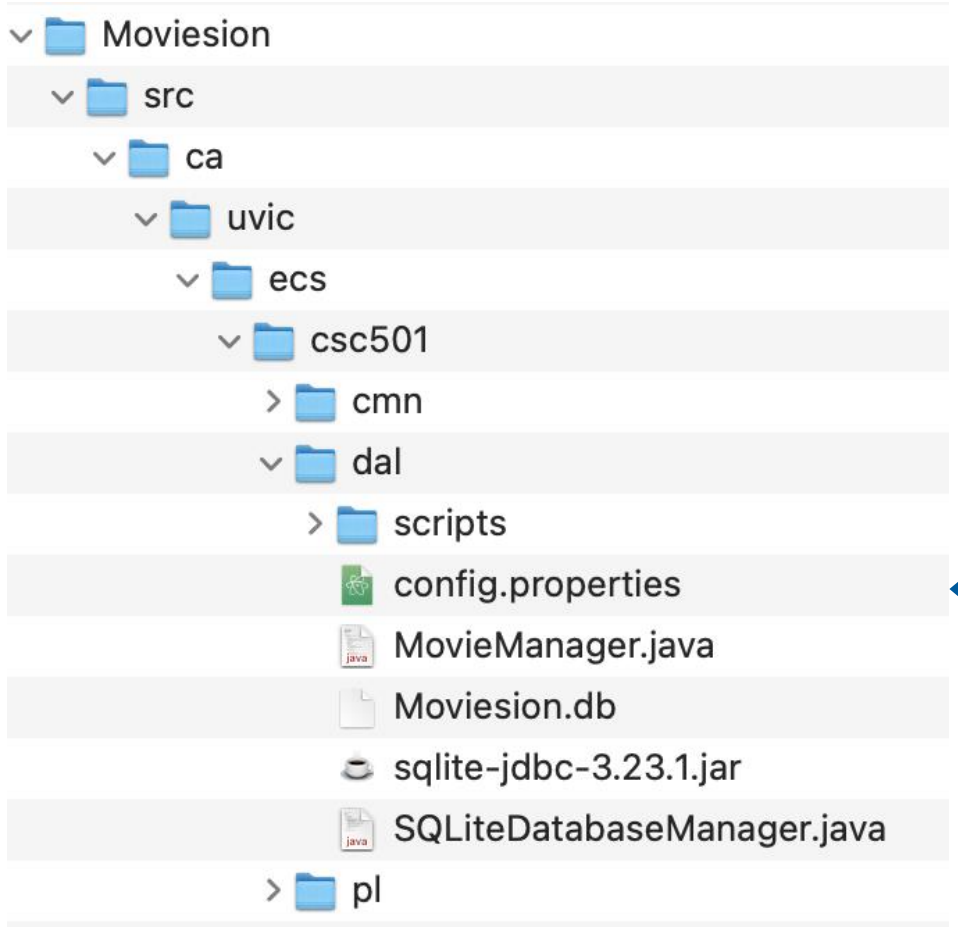
47

# SQL: DML via DAL




- Moviesion
  - src
    - ca
      - uvic
        - ecs
          - csc501
            - cmn
              - Director.java
              - Movie.java
            - dal
              - scripts
                - adhoc
                - schema.sql
              - config.properties
              - MovieManager.java
              - Moviesion.db
              - sqlite-jdbc-3.23.1.jar
              - SQLiteDatabaseManager.java
            - pl
              - Moviesion.java

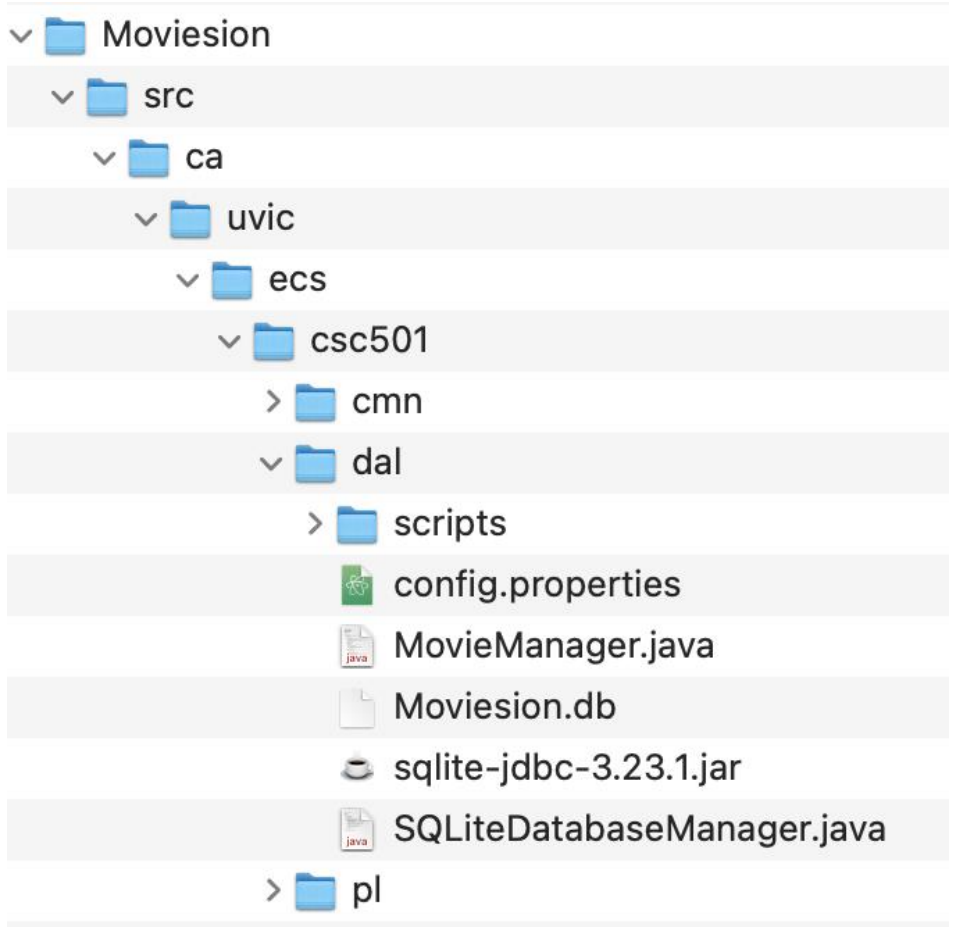
# SQL: DML via DAL




1

Settings to connect to RDBMS:
Location, Username, Password, DatabaseName

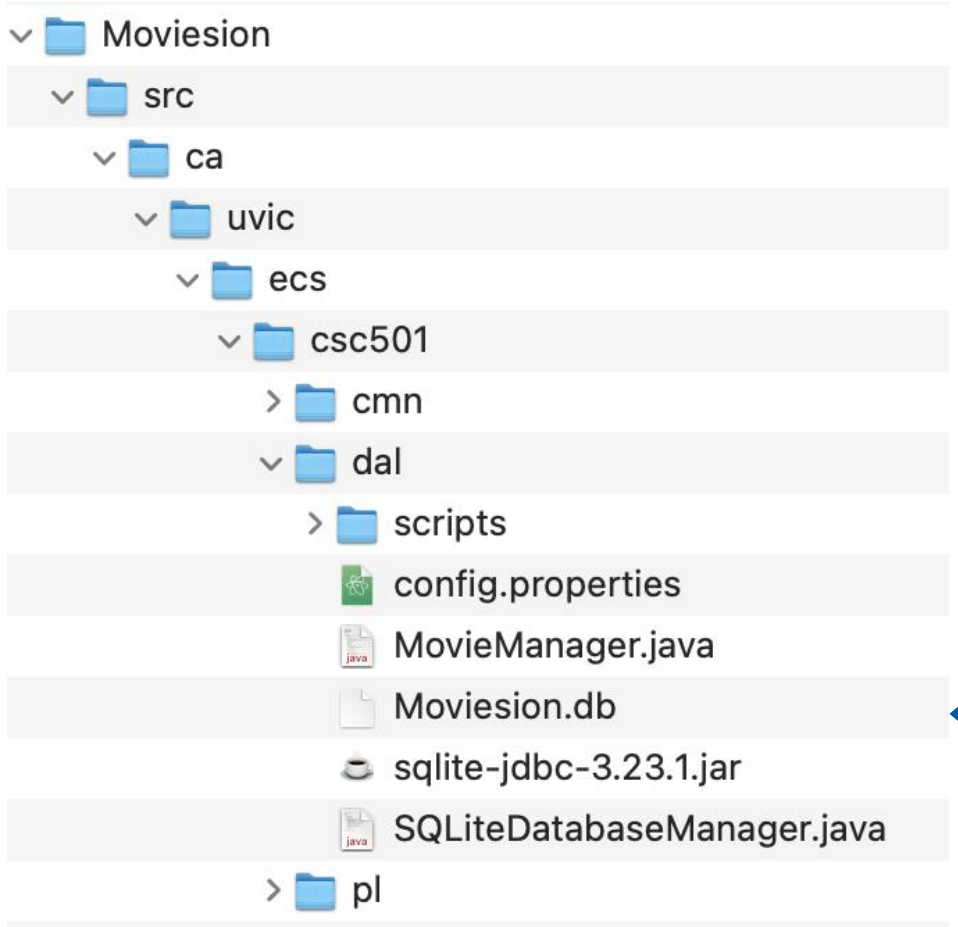
# SQL: DML via DAL




2

Anything related to DBMS connection:
Loading SQLite library
Loading settings from config file
Create a connection

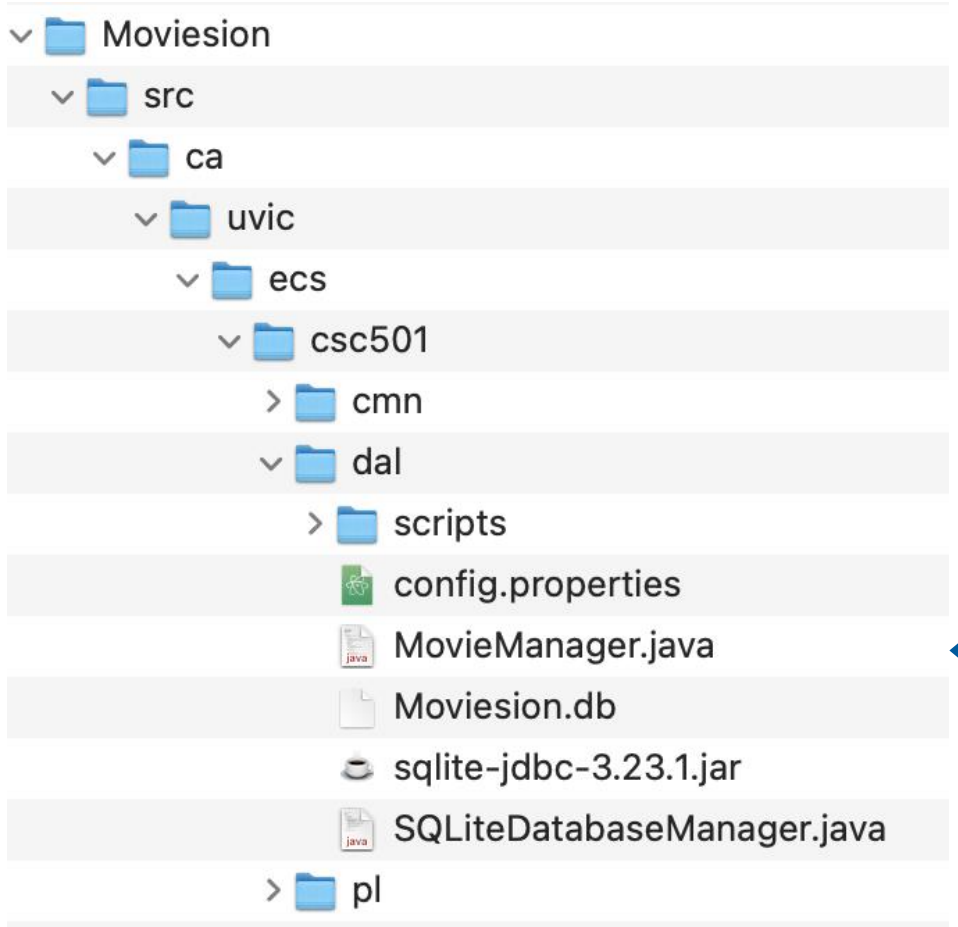
# SQL: DML via DAL




3

The database, including all relations.

# SQL: DML via DAL




4

Anything related to Movie relation (table):

| | |
|---|---|
| addMovie() | INSERT |
| editMovie() | UPDATE |
| removeMovie() | DELETE |
| getMovie() | SELECT |